

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“Jnana Sangama”, Belgaum-590 014, Karnataka.



A Mini Project Report On

“SAHAYAKA - AI Citizen Grievance Redressal System”

Submitted in the partial fulfilment of the requirements for the award of the Degree of

BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND ENGINEERING

Submitted by

**Punith A
1MP23CS045**

**Rohith Gowda R
1MP23CS049**

**Yagna Maurya M V
1MP23CS061**

Under the Guidance of

**Dr. Manjula G
Professor & Head, Dept of CSE**



**BGS COLLEGE OF ENGINEERING (BGSCET)
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

BANGALORE - 560 086

2025-2026

BGS COLLEGE OF ENGINEERING AND TECHNOLOGY

(Approved by AICTE, New Delhi and Affiliated to VTU, Belagavi)

Adjacent to Mahalakshmi Metro Station, Mahalakshmiapuram, West of Chord Road, Bengaluru -560 086

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the mini project work entitled “**SAHAYAKA-AI Citizen Grievance Redressal System**” presented by **Punith A(IMP23CS045)**, **Rohith Gowda R(IMP23CS049)**, **Yagna Maurya M V(IMP23CS061)**, Bonafide students of **BGS COLLEGE OF ENGINEERING AND TECHNOLOGY**, Bangalore in partial fulfilment for the award of **Bachelor of Engineering in Computer Science** in **Visvesvaraya Technological University**, Belgaum during the year **2025-2026**. It is certified that all corrections/suggestions indicated have been incorporated in the report. The mini-project has been approved as it satisfies the academic requirements in respect of the mini project prescribed for the said degree.

Dr. Manjula G

Prof & Head, Dept. of CSE

BGSCET, Bangalore

Dr. Manjula G

Prof & Head, Dept. of CSE

BGSCET, Bangalore

Dr. Ravikumar GK

Principal

BGSCET, Bangalore

ACKNOWLEDGEMENT

We bow in deep reverence and gratitude to His Divine Soul **Jagadguru Padmabhushan Sri Sri Sri Dr. Balagangadharanatha Mahaswamiji** and His Holiness **Jagadguru Sri Sri Sri Dr. Nirmalanandanatha Mahaswamiji**, whose divine grace and blessings have enabled us to pursue and successfully complete our academics in this esteemed institution.

We would also like to express our humble gratitude to **Sri Sri . Soumyanatha Swamiji** for his blessings and encouragement, which have been a source of inspiration in our academic journey.

We wish to express our deep sincere feelings of gratitude to our institution, BGS College of Engineering & Technology, for providing an opportunity for completing our Project work successfully.

We would also like to express our deep sense of gratification to **Dr. Raju G T**, Director, BGS College of Engineering & Technology, for his continuous support in providing amenities to carry out this Project work in this admired institution.

We express our deep appreciation to **Dr. RaviKumar G K**, Principal, BGS College of Engineering & Technology, for providing us with excellent facilities and academic ambience; which have helped us in satisfactory completion of Project work.

We extend our sincere thanks to **Dr. Manjula G**, Head of the Department, Computer Science and Engineering for providing us an invaluable support throughout the period of our Project work.

We wish to express our heartfelt gratitude to our Project Coordinator **Dr. Manjula G**, Head of the Department, CSE for her valuable guidance, suggestions and cheerful encouragement during the entire period of our Project work.

Finally, we take this opportunity to extend our earnest gratitude and respect to our parents, Teaching & Non teaching staff of the department, the library staff and all our friends, who have directly or indirectly supported us during the period of our Project work.

Regards,

Punith A (IMP23CS045)

Rohith Gowda R(IMP23CS049)

Yagna Maurya MV(IMP23CS061)

Abstract

The increasing demand for efficient and transparent public service delivery has highlighted the limitations of traditional grievance redressal mechanisms, which often suffer from delays, manual processing inefficiencies, and inconsistent routing of complaints. This paper presents the design and implementation of an **AI-based Grievance Redressal System** that leverages Natural Language Processing (NLP), Machine Learning (ML), and Large Language Models (LLMs) to automate the end-to-end lifecycle of complaint management. The system enables citizens to interact with an AI-driven assistant capable of understanding natural language inputs, extracting key information, categorizing grievances, and generating unique reference identifiers.

The proposed architecture integrates a Node.js backend for authentication, workflow coordination, and media handling with a Python FastAPI microservice hosting an LLM-based grievance analysis engine. A hybrid storage model using MongoDB and SQLite ensures scalable and structured management of user data, complaints, tickets, evidence, and session logs. Role-based dashboards for citizens, administrators, and department officers provide real-time tracking, automated routing, SLA monitoring, and SMS notifications. Experimental evaluation demonstrates that the AI-assisted workflow significantly reduces complaint processing time, improves routing accuracy, and enhances user engagement through conversational interfaces. Future improvements include speech-enabled grievance lodging, real-time analytics, and integration with blockchain for secure and auditable complaint handling.

Contents

List of Figures	1
1 Introduction	2
1.1 Introduction to AI-Based Grievance Redressal System	2
1.2 Introduction to Artificial Intelligence (AI)	3
1.3 History of Artificial Intelligence	5
1.4 Definition of Artificial Intelligence	6
1.5 Uses of Artificial Intelligence	7
1.6 Advantages of Artificial Intelligence	8
2 Requirement Specification	9
2.1 Hardware Requirements	9
2.2 Software Requirements	11
3 Description of Tools and Technologies	14
3.1 Programming Languages	14
3.2 Frontend Technology	15
3.3 Backend Technologies	15
3.4 Database Technologies	16
4 System Architecture	17
4.1 Architecture Overview	17
4.2 Functional Flow	17

4.3 Data Flow Diagrams	18
4.4 Security Design	18
4.5 Design Diagrams	19
5 Testing	23
5.1 Testing Strategy	23
5.2 AI-Specific Testing Metrics	23
5.3 Test Cases	24
6 Implementation and Code	25
6.1 Implementation Logic: The RAG Pipeline	25
6.2 Python (FastAPI) — AI Classification Endpoint	25
6.3 Node.js (Express) — Handling Submission	27
6.4 Deployment Strategy	28
7 Snapshots	29
Conclusion	32

List of Figures

4.1	Use Case Diagram for AI Grievance Redressal System	19
4.2	ER / Database Schema Diagram of the AI Grievance Redressal System	20
4.3	Class Diagram showing core entities and relationships	21
4.4	System Architecture Diagram showing client, backend, AI and database layers	21
4.5	Tier-based Architecture of the AI Grievance Management System	22
7.1	Citizen Dashboard showing welcome message and current tickets	29
7.2	AI Chatbot interface collecting grievance details from the citizen	30
7.3	Chatbot confirming grievance creation with reference details	30
7.4	Admin Dashboard with recent complaints and status summary	31
7.5	Citizen view of all complaints raised with their current status	31

Chapter 1

Introduction

The advent of the digital age has fundamentally reshaped the expectations of citizens and consumers regarding their interactions with governing bodies and service providers. In this contemporary landscape, the efficiency, transparency, and responsiveness of grievance redressal mechanisms have emerged as critical benchmarks for organizational integrity and public trust. An Artificial Intelligence (AI)-based Grievance Redressal System represents a paradigm shift from traditional, often reactive, complaint management methodologies to a proactive, intelligent, and data-driven framework.

1.1 Introduction to AI-Based Grievance Redressal System

Unlike conventional systems that rely heavily on manual intervention—often resulting in bureaucratic bottlenecks, misclassification of issues, and significant delays—an AI-driven architecture leverages advanced computational capabilities to automate, streamline, and optimize the entire lifecycle of a grievance.

The core proposition of this project is to develop a sophisticated platform that utilizes Machine Learning (ML), Natural Language Processing (NLP), and Large Language Models (LLMs) to bridge the communication gap between the complainant and the administration. By deploying these technologies, the system can autonomously parse unstructured complaint text, identify the underlying intent and sentiment, categorize the grievance into specific administrative domains (e.g., sanitation, finance, human resources), and route it to the appropriate authority with high precision.

This automation is not merely administrative; it is transformative. It democratizes access to justice by enabling features such as multilingual support, where a complaint lodged in a local dialect is automatically translated and summarized for the redressal officer, ensuring that language barriers do not impede the resolution process.

Furthermore, the system integrates predictive analytics to move governance from a responsive stance to a preventative one. By analyzing historical data and identifying clusters of similar grievances, the AI can alert authorities to systemic failures—such as a recurring infrastructure defect in a specific locality or a widespread service outage—allowing for intervention before

minor issues escalate into major public crises. This project, therefore, aims to create a robust, scalable, and inclusive grievance redressal ecosystem that enhances operational efficiency, reduces the cognitive load on human officers, and significantly improves user satisfaction through timely and empathetic engagement.

Key Points to Cover

- **Purpose:** The fundamental purpose is to alleviate the administrative burden of manual complaint sorting and to ensure that every grievance, regardless of its origin or language, is acknowledged, categorized, and addressed with speed and accuracy. It aims to replace opaque processes with a transparent, trackable, and accountable system.
- **Use Cases:** The versatility of this AI tool allows for deployment across diverse sectors. In E-Governance, it can manage civic issues ranging from waste management to public transport irregularities, as seen in initiatives like CPGRAMS. In the Banking and Financial Services sector, it acts as a first-line defense for dispute resolution, adhering to strict regulatory frameworks and identifying potential fraud through anomaly detection. In Corporate Environments, it provides a confidential channel for employee grievances, ensuring unbiased handling of HR issues.
- **Impact:** The deployment of such a system significantly enhances the quality of service delivery. It reduces the turnaround time (TAT) for complaint resolution, minimizes human error in routing, and fosters a culture of continuous improvement by providing actionable insights derived from user feedback trends.

1.2 Introduction to Artificial Intelligence (AI)

Artificial Intelligence (AI) constitutes a broad and transformative branch of computer science dedicated to the creation of systems capable of performing tasks that historically required human intelligence. These tasks encompass a wide spectrum of cognitive abilities, including visual perception, speech recognition, decision-making, and the translation of languages. In the specific context of an automated grievance redressal system, AI functions not merely as a tool for mechanization but as a cognitive engine capable of interpreting the nuances of human distress, understanding complex administrative hierarchies, and generating context-aware responses.

The evolution of AI from simple rule-based algorithms to complex neural networks has enabled the development of systems that can “learn” from data rather than following strictly

pre-programmed instructions. This capability is central to the project, as the system must adapt to the evolving nature of public grievances and the changing vernacular of the user base.

Key Components

- **Machine Learning (ML):** ML is the foundational subset of AI that focuses on the development of algorithms that improve automatically through experience. In this project, supervised learning models are trained on historical grievance datasets to predict the category and severity of incoming complaints. For instance, a model might learn to associate keywords like “pothole”, “road”, and “accident” with the “Public Works Department” and assign a high-urgency tag.
- **Natural Language Processing (NLP):** NLP is the critical interface between human language and machine understanding. It enables the system to parse the unstructured text of a complaint, identifying key entities such as names, dates, locations, and specific problem descriptions (Named Entity Recognition). NLP techniques also facilitate sentiment analysis, allowing the system to gauge the emotional state of the complainant and prioritize the grievance accordingly.
- **Computer Vision:** While the primary mode of interaction is textual, modern grievance systems often allow for the upload of photographic evidence. Computer vision algorithms can analyze these images to verify the complaint (e.g., detecting a garbage pile in an uploaded photo) and assess the severity of the issue, adding a layer of automated verification to the process.
- **Neural Networks and Deep Learning:** Deep learning, inspired by the neural structure of the human brain, powers the most advanced capabilities of the system. Large Language Models (LLMs) based on Transformer architectures utilize deep learning to generate coherent, empathetic summaries of long-winded complaints and draft appropriate responses, mimicking the cognitive sophistication of a human caseworker.
- **AI in Healthcare and Governance:** The principles applied here parallel those in healthcare AI, where systems analyze medical reports to assist doctors. Similarly, in governance, AI assists administrators by synthesizing vast amounts of citizen data into actionable intelligence, transforming the efficiency of public service delivery.

1.3 History of Artificial Intelligence

The historical trajectory of Artificial Intelligence is a narrative of ambitious theoretical beginnings, periods of disillusionment, and a modern renaissance driven by data and computational power. Understanding this history is essential to appreciate the technological maturity that makes an AI Grievance Redressal System feasible today.

Early Concepts (1950s–1960s)

The conceptual foundations of AI were laid in the mid-20th century. Alan Turing’s seminal 1950 paper, *Computing Machinery and Intelligence*, posed the provocative question, “Can machines think?” and introduced the Turing Test as a criterion for intelligence. The field was formally crystallized in 1956 at the Dartmouth Conference, where John McCarthy coined the term “Artificial Intelligence”. During this period, early programs like the Logic Theorist and the General Problem Solver demonstrated that machines could perform symbolic reasoning and solve logic puzzles. However, these systems were brittle, operating effectively only in strictly controlled micro-worlds, and lacked the robustness required for real-world problem solving.

The AI Winter (1970s–1980s)

The initial euphoria gave way to the “AI Winter” in the 1970s, characterized by funding cuts and skepticism as researchers realized the immense difficulty of replicating general human intelligence and common sense. A resurgence occurred in the 1980s with the advent of “Expert Systems”. These systems utilized rule-based engines to mimic the decision-making abilities of a human expert in specific domains (e.g., medical diagnosis or geological exploration). In the context of grievance redressal, early automated phone systems (IVR) can be seen as primitive descendants of this era—rigid, menu-driven interfaces that frustrated users and often failed to resolve complex issues.

The Rise of Machine Learning and Big Data (1990s–2000s)

The 1990s marked a pivotal shift from knowledge-driven approaches (encoding rules) to data-driven approaches (learning from examples). The explosion of the internet generated vast datasets, fueling the development of statistical machine learning algorithms. Techniques like Support Vector Machines (SVMs) and Random Forests became standard for classification tasks, such as filtering spam emails—logic that is directly applicable to categorizing grievances today.

Modern AI (2010s–Present)

The current era is defined by the dominance of Deep Learning. The convergence of massive datasets, powerful GPU hardware, and algorithmic breakthroughs allowed AI to surpass human performance in image and speech recognition. The introduction of the Transformer architecture in 2017 revolutionized NLP, paving the way for Generative AI and Large Language Models. These modern models possess a nuanced understanding of context and semantics, enabling the creation of the conversational, empathetic, and intelligent grievance redressal system proposed in this report.

1.4 Definition of Artificial Intelligence

To establish a precise theoretical framework for this project, it is necessary to define Artificial Intelligence both in its classical sense and in the context of its modern application.

Turing’s Definition (1950)

Alan Turing avoided a direct definition of “thinking”, instead proposing an operational test: “The question is not, ‘Can machines think?’, but ‘Can machines do what we (as thinking entities) can do?’ ” This functionalist definition underpins the goal of the grievance system—it matters less if the system “understands” the grievance in a human sense, and more that it processes and resolves it as competently as a human officer.

Modern Definition

AI is defined as the simulation of human intelligence processes by machines, particularly computer systems. These processes include learning (the acquisition of information and rules for using the information), reasoning (using rules to reach approximate or definite conclusions), and self-correction. Within this project, AI is specifically defined as the computational ability to ingest unstructured human language, extract semantic meaning, classify the content based on predefined administrative taxonomies, and generate intelligent, context-aware responses without continuous human oversight.

Key Points to Cover

- **Intelligence Simulation:** The system simulates the cognitive workflows of a grievance officer: reading a complaint, assessing its urgency, checking against policy rules (Reasoning), and deciding the routing path (Decision Making).
- **Cognitive Functions:** It replicates specific cognitive functions such as perception (interpreting text and images via OCR and NLP) and communication (drafting coherent replies in natural language).

1.5 Uses of Artificial Intelligence

While this report focuses on grievance redressal, acknowledging the pervasive application of AI across various domains highlights the maturity and versatility of the technologies being employed.

Key Points to Cover

- **Healthcare:** AI is transforming healthcare by analyzing medical reports, aiding in diagnostics (e.g., detecting tumors in radiology scans), personalized medicine, and predictive analytics for patient deterioration. The NLP techniques used to summarize clinical notes are identical to those used for summarizing citizen grievances in this project.
- **Finance:** In the financial sector, AI algorithms detect fraudulent transactions in milliseconds and power high-frequency algorithmic trading. In the context of grievance redressal, similar anomaly detection algorithms are used to identify spam, duplicate, or malicious complaints that clog the system.
- **Business:** Chatbots, virtual assistants, and recommendation engines drive customer engagement and sales. The conversational agents used in this project are an evolution of these business tools, adapted for civic engagement rather than commerce.
- **Transportation:** Autonomous vehicles and intelligent traffic management systems utilize AI to optimize flow and safety, demonstrating AI's capability to handle real-time, critical decision-making.
- **AI in Medical Report Summarization:** AI algorithms parse through long, technical medical documents to extract key insights. This mirrors the grievance system's

function of parsing detailed complaint letters to extract actionable data points like “Location”, “Category”, and “Urgency”.

1.6 Advantages of Artificial Intelligence

The integration of AI into grievance redressal offers transformative benefits that manual systems cannot match, addressing the “Iron Triangle” of speed, cost, and quality.

Key Points to Cover

- **Efficiency:** AI processes complaints in milliseconds. What would take a human officer several minutes to read, comprehend, and categorize is done instantly. This drastic reduction in processing time is crucial for clearing the massive backlogs often found in public administration.
- **Accuracy:** Human fatigue and bias often lead to errors in categorization or routing (e.g., sending a water supply complaint to the electricity department). AI systems, once rigorously trained, maintain consistent accuracy, ensuring that grievances reach the correct authority the first time.
- **Automation:** Repetitive, low-value tasks such as sending acknowledgement emails, checking complaint status, and routing common queries are fully automated. This allows human officers to focus their cognitive resources on complex, high-priority cases that require empathy and judgment.
- **Cost Efficiency:** By automating the initial tiers of support, organizations can handle a significantly higher volume of grievances without a linear increase in staffing costs. Operational costs related to paper handling, manual data entry, and call center staffing are substantially reduced.
- **Personalized Care and Inclusivity:** AI enables personalized interactions at scale. It can address citizens by name, reference their past history, and communicate in their preferred language. This is particularly vital for inclusivity, as voice-enabled AI allows illiterate or differently-abled citizens to interact with the government independently.
- **Scalability:** During crises (e.g., natural disasters or service outages), grievance volumes can spike exponentially. AI systems scale instantly to handle thousands of concurrent requests, a feat impossible for human-staffed call centers.

Chapter 2

Requirement Specification

This chapter outlines the comprehensive hardware and software requirements necessary to build, deploy, and maintain the AI-based Grievance Redressal System. These specifications are designed to support the intensive computational demands of Large Language Models (LLMs) and the high concurrency expected in a public-facing application.

2.1 Hardware Requirements

The hardware infrastructure must be robust enough to support the full pipeline: from the development environment where models are fine-tuned and tested, to the production environment where they serve thousands of users. The requirements are driven by the need for low-latency inference (for real-time chat) and high throughput (for processing bulk complaints).

Central Processing Unit (CPU)

- **Processor Type:** A high-performance multi-core processor is essential. While the GPU handles the heavy lifting of model inference, the CPU manages data preprocessing, API request routing, and database interactions.
- **Development:** An Intel Core i7 (10th Gen or newer) or AMD Ryzen 7 (5000 series or newer) is recommended. These processors offer high single-thread performance, which is beneficial for the Node.js runtime and development tools.
- **Deployment Server:** Server-grade processors like Intel Xeon Scalable or AMD EPYC are recommended. These CPUs support advanced vector instructions that can accelerate certain tensor operations if CPU-only inference fallback is required.

Memory (RAM)

- **Minimum RAM:** 16 GB is the absolute minimum required to run the operating system, the Docker containerization layer, and the development stack (Node.js, Python, Database) simultaneously.

- **Recommended RAM:** 32 GB or higher is strongly advised. Running local LLMs requires substantial memory to load the model weights and maintain the context window (conversation history).
- **High-Performance RAM:** For production servers handling high concurrency, 64 GB ECC (Error Correcting Code) RAM is recommended to ensure system stability and prevent data corruption during continuous operation.

Storage

- **Solid State Drive (SSD):** The use of NVMe SSDs is essential. Loading massive AI models into memory and querying large databases of grievances requires high I/O throughput.
- **Capacity:** A minimum of 512 GB is required for the OS, software dependencies, and model weights (which can range from a few GB to tens of GB depending on model choice).
- **Backup Storage:** Integration with cloud object storage services (like AWS S3, Google Cloud Storage, or Azure Blob Storage) is required for securely storing uploaded media (images/videos of grievances) and regular database backups.

Graphics Processing Unit (GPU)

- **Importance:** Efficient and fast inference of LLMs dictates the use of a dedicated GPU. While CPU inference is possible, it is often too slow for a real-time conversational interface.
- **Minimum GPU:** An NVIDIA GPU with at least 8 GB VRAM (e.g., RTX 3060 or RTX 4060). This allows for running quantized versions of smaller models comfortably.
- **Recommended GPU:** For optimal performance, especially if fine-tuning is required or if multiple concurrent inference streams are needed, NVIDIA RTX 3090/4090 (24 GB VRAM) or data-center grade cards like the A100 or H100 are recommended.
- **CUDA Support:** The GPU must support recent CUDA versions to interface effectively with deep learning libraries such as PyTorch and TensorFlow.

Network Connectivity

- **Internet Connection:** High-speed, low-latency internet is required. This is essential for making API calls to cloud services (if using cloud-based LLMs as a fallback), downloading model updates, and serving the web application to end-users without latency.
- **Local Network:** If deployed within a secure government intranet, a high-bandwidth local network is preferred to facilitate fast data transfer between the application server, the database server, and the storage server.

Peripheral Devices

- High-resolution monitors (1080p or 4K) for developers.
- Standard keyboard and mouse for interaction.
- Printer/scanner for scanning physical grievance letters to digitize them using the system's OCR capabilities.

2.2 Software Requirements

The software stack is designed as a sophisticated hybrid architecture, leveraging the strengths of different technologies to ensure performance, scalability, and ease of development.

Frontend (Client-Side)

- **Framework:** React.js is chosen for its component-based architecture and Virtual DOM, which ensures a highly responsive user interface. It efficiently manages the state of complex grievance forms and the real-time updates required for the chat interface.
- **UI Library:** Tailwind CSS or Bootstrap for rapid, responsive styling that adapts seamlessly to mobile and desktop screens.
- **State Management:** Redux or Context API to manage global application state, such as user authentication status and the progress of multi-step grievance forms.

Backend (Server-Side — Hybrid Architecture)

- **Primary Backend:** Node.js with Express.js. Node.js is selected for its non-blocking, event-driven architecture, ideal for handling I/O-bound tasks like thousands of concurrent HTTP requests, serving static files, and managing WebSocket connections for real-time chat.
- **AI Microservice:** Python with FastAPI. Python is the dominant language of AI. FastAPI wraps the AI models into high-performance REST endpoints and supports asynchronous processing, allowing multiple inference requests in parallel.
- **Integration:** The Node.js server acts as the API Gateway, routing user requests to the Python microservice for intelligent processing and aggregating the results for the frontend.

Database

- **Primary Database:** MongoDB, a NoSQL database, is chosen due to the unstructured nature of grievance data and chat logs.
- **Vector Database:** ChromaDB or FAISS is used to enable Retrieval Augmented Generation (RAG), allowing fast similarity searches on high-dimensional vector embeddings of policies and past grievances.

AI Models and Libraries

- **LLM:** Llama-based local models via Ollama for privacy-preserving local deployment, or cloud-based models such as Google Gemini for higher reasoning and large context windows.
- **Orchestration:** Libraries such as LangChain are used for chaining components (retrieval, prompt construction, and generation).
- **OCR:** Tesseract or EasyOCR for extracting text from scanned documents and images uploaded by users.

Infrastructure & Deployment

- **Containerization:** Docker to package Node.js, Python services, and MongoDB into isolated containers.

- **Orchestration:** Docker Compose or Kubernetes for managing multi-container deployment and scaling.

Features & User Flow

Key user-facing features include:

- Homepage with overview and announcements.
- Grievance submission via multi-modal form (text, voice, images).
- Real-time chatbot for guidance and status queries.
- Multilingual support with automatic translation.
- Admin dashboard for tracking trends and performance metrics.

Error Handling & Security

- **Retry Logic:** Automatic retries with exponential backoff for AI service failures.
- **Fallback Analysis:** Rule-based keyword matching if the primary LLM is unavailable.
- **Security:** API keys and database credentials are managed using environment variables; all communications use HTTPS.

Chapter 3

Description of Tools and Technologies

The selection of tools for the AI Grievance Redressal System is a strategic decision, balancing rapid development, high performance, and the integration of cutting-edge AI capabilities.

3.1 Programming Languages

Python

Python serves as the engine room for the project's Artificial Intelligence capabilities. Its selection is driven by:

- A rich ecosystem of AI and ML libraries such as PyTorch, TensorFlow, and Hugging Face Transformers.
- Simplicity and readability, enabling rapid prototyping of complex algorithmic logic like the RAG pipeline.
- Powerful data processing libraries such as Pandas and NumPy for analytics and dashboard data preparation.

JavaScript (Node.js)

JavaScript is utilized for full-stack web development.

- **Unified Stack:** JavaScript on both client (React) and server (Node.js) allows code reuse and simplifies development.
- **Performance:** Node.js's non-blocking I/O model is optimized for real-time applications with many concurrent users.

3.2 Frontend Technology

React.js

React is a declarative, efficient, and flexible JavaScript library for building user interfaces.

- Component-based architecture for reusable UI elements such as `ComplaintForm`, `StatusTracker`, and `ChatWindow`.
- Virtual DOM for efficient rendering and smooth user experience.
- Rich ecosystem including React Router, Redux, and React Hook Form.

Streamlit (for Admin Analytics)

While React is the primary portal for citizens, Streamlit can be used for rapid development of admin analytics dashboards and AI prototypes, enabling easy visualization of grievance heatmaps, sentiment distributions, and model performance.

3.3 Backend Technologies

Node.js with Express

Node.js with Express serves as the central orchestration layer or API Gateway.

- Handles user authentication, static assets, and file uploads.
- Routes analysis requests to Python services.
- Uses middleware for logging, error handling, and security headers.

FastAPI

FastAPI is a modern, high-performance framework for building APIs in Python.

- High performance with asynchronous support.

- Built-in data validation using Pydantic.
- Ideal for wrapping AI models into REST endpoints.

LLM Technologies

- **Cloud Models (e.g., Gemini):** Used when high reasoning capacity or large context windows are required.
- **Local Models (e.g., Llama via Ollama):** Used when data privacy or offline operation is required.

3.4 Database Technologies

MongoDB

A document-oriented NoSQL database that provides flexible schema to store heterogeneous grievances, metadata, and chat histories.

Vector Store

ChromaDB or FAISS is used to store embeddings of policies, FAQs, and historical grievances. This enables semantic search within the RAG pipeline.

Chapter 4

System Architecture

The AI Grievance Redressal System is architected as a Hybrid Microservices Application. This design decouples the high-concurrency web server from the computation-heavy AI processing unit.

4.1 Architecture Overview

The system is stratified into three logical layers:

1. **Presentation Layer (Client):** React-based interface for citizens and administrators.
2. **Application Layer (Server):** Node.js gateway and Python AI service.
3. **Data Layer:** MongoDB and a vector store (ChromaDB/FAISS).

Architecturally, Node.js handles I/O-bound workloads, while Python manages math-heavy AI computations. They communicate over a secure internal network, combining concurrency with computational power.

4.2 Functional Flow

Complaint Lodging

- Users log in via the React frontend and submit grievances via text, images, or voice.
- Node.js validates the session and stores raw data in MongoDB with a “Pending” status.

AI Analysis

- OCR extracts text from images where needed.
- The combined text is classified into department, urgency, and sentiment.

- The text is embedded and passed through the RAG pipeline to retrieve relevant policies and similar past cases.
- The LLM generates a summary for officers and a preliminary response for the citizen.

Routing and Resolution

- Node.js receives the AI output and updates the grievance record.
- The complaint is routed to the appropriate officer dashboard.
- Officers review AI suggestions, finalize responses, and close the ticket. Citizens are notified via SMS/email.

4.3 Data Flow Diagrams

Level 0 DFD (Context Diagram)

- External entities: Citizen, Grievance Officer, Administrator.
- Central process: AI Grievance Redressal System.
- Data flow: Citizens submit complaints; the system returns acknowledgments and status; officers handle routed complaints; administrators receive analytics reports.

4.4 Security Design

Given the sensitive nature of grievance data, security is paramount:

- **Data Encryption:** All data in transit is encrypted via HTTPS; data at rest is encrypted using industry-standard algorithms.
- **PII Redaction:** A local NLP module redacts personally identifiable information before sending text to any cloud-based AI service.
- **Role-Based Access Control (RBAC):** Officers can only access grievances relevant to their department.
- **Rate Limiting:** API rate limiting prevents Denial of Service (DoS) attacks.

4.5 Design Diagrams

This section presents the major design diagrams used to model the system from different perspectives.

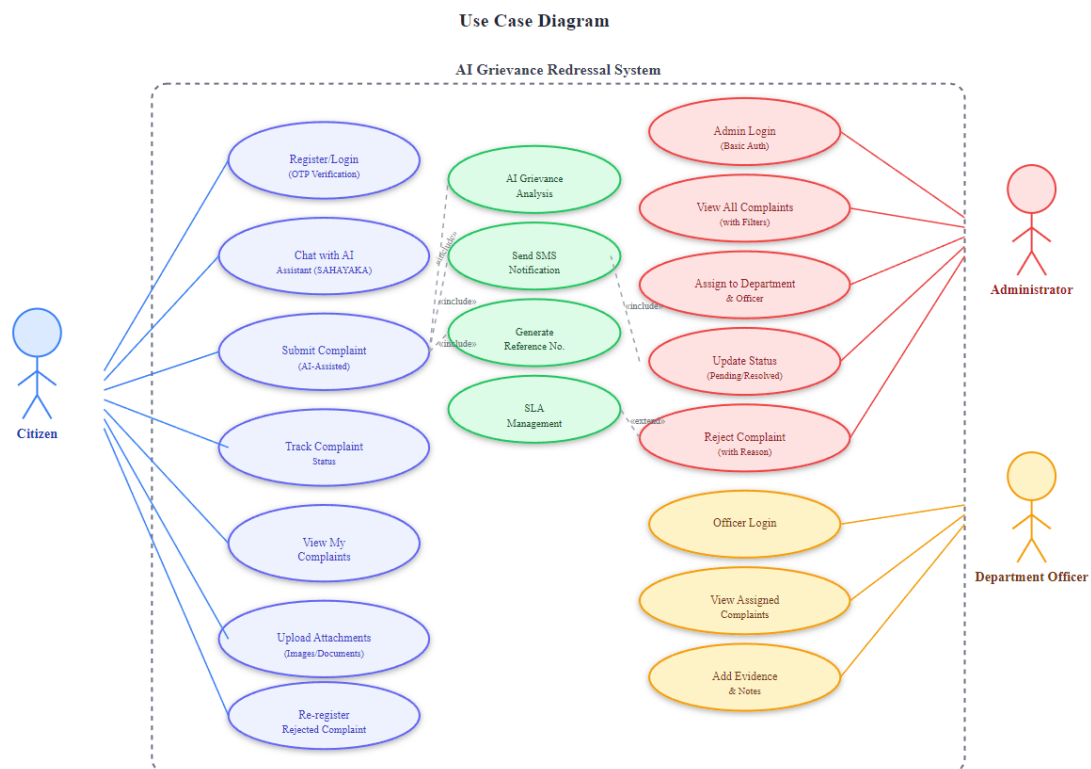


Figure 4.1: Use Case Diagram for AI Grievance Redressal System

ER / Database Schema Diagram

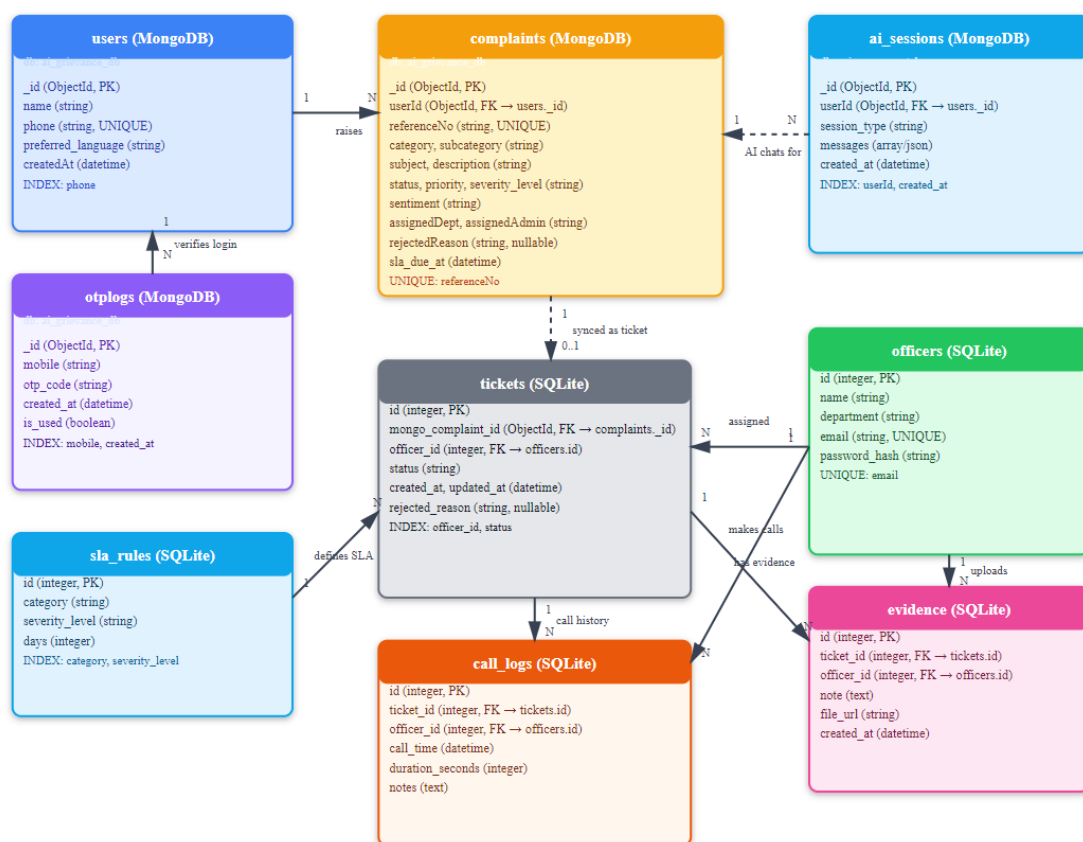


Figure 4.2: ER / Database Schema Diagram of the AI Grievance Redressal System

Class Diagram

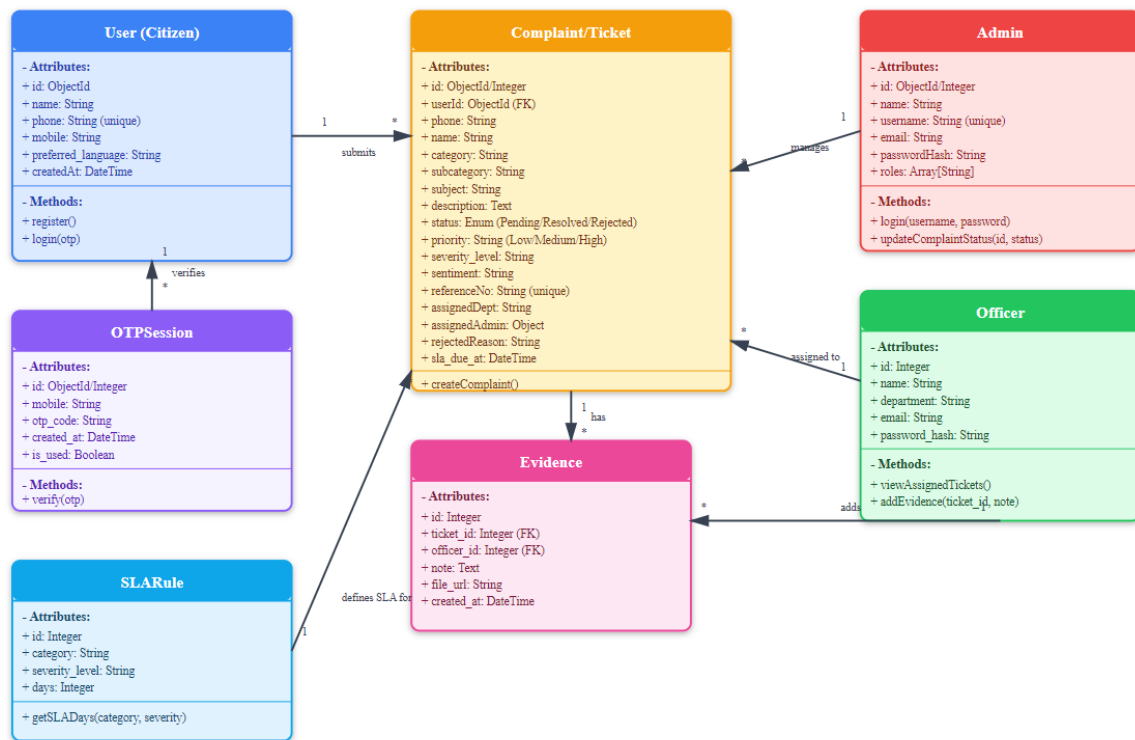


Figure 4.3: Class Diagram showing core entities and relationships

System Architecture Diagram

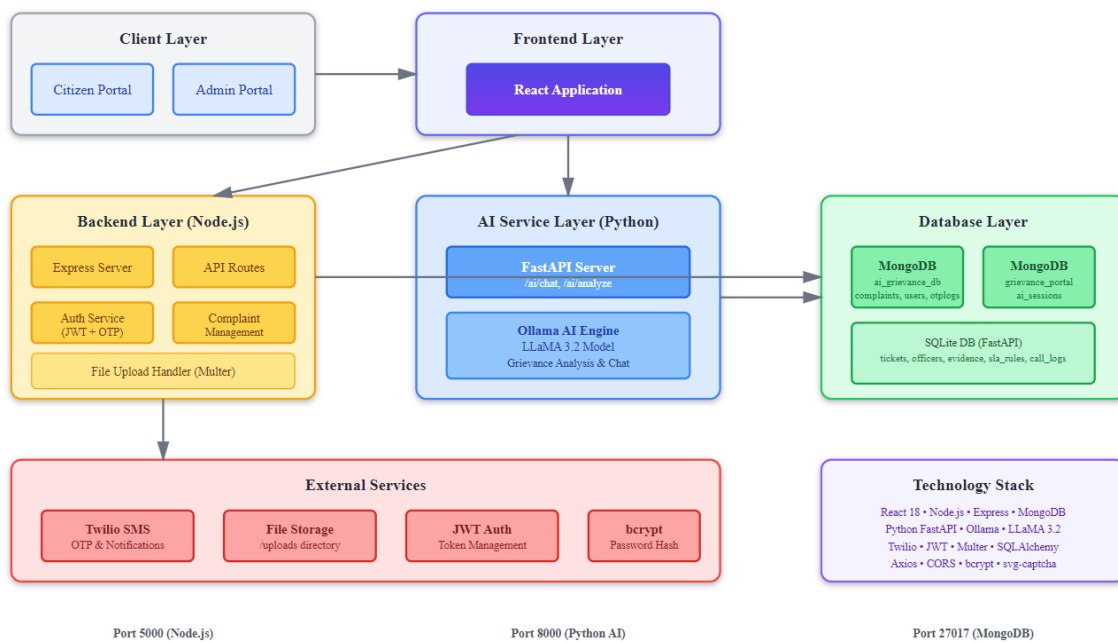


Figure 4.4: System Architecture Diagram showing client, backend, AI and database layers

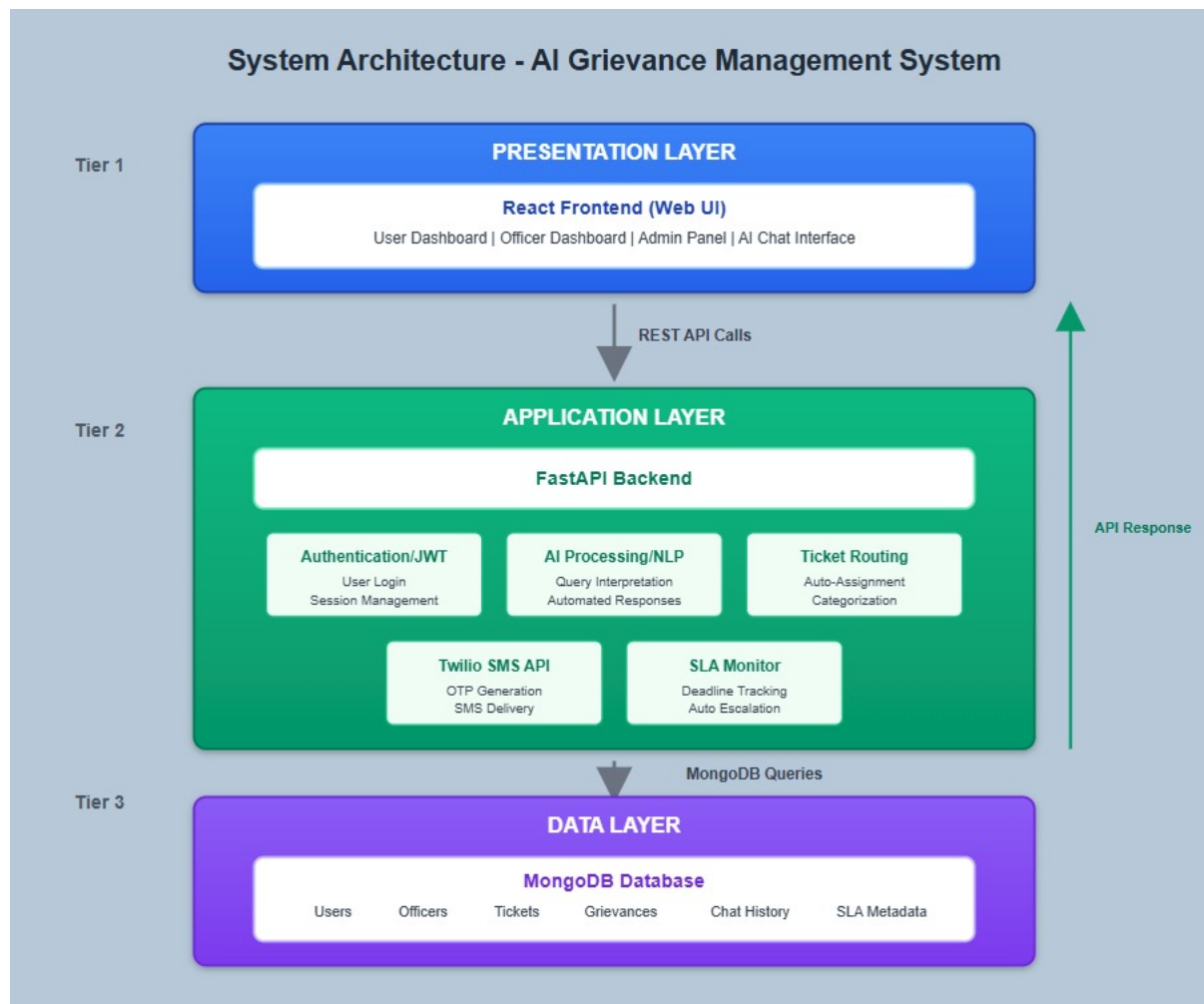


Figure 4.5: Tier-based Architecture of the AI Grievance Management System

Chapter 5

Testing

Testing an AI-based system requires both deterministic software testing and probabilistic AI evaluation.

5.1 Testing Strategy

Unit Testing

- Tools: Jest for Node.js and React, `pytest` for Python.
- Scope: Individual functions such as file validators, API wrappers, and fallback logic.

Integration Testing

- Verifies the interaction between Node.js and FastAPI services, including JSON payload compatibility and correct storage in MongoDB.

System Testing (End-to-End)

- Scenario: A user uploads a grievance, the system processes and classifies it, and an officer resolves it via the dashboard.

5.2 AI-Specific Testing Metrics

AI evaluation metrics include:

- **Answer Relevancy:** Measures how much of the AI response is directly related to the grievance.
- **Contextual Recall:** Ability of RAG to retrieve correct laws and past rulings.

- **Hallucination Rate:** Frequency of factually incorrect or fabricated information.
- **Sentiment Accuracy:** Alignment between model-detected sentiment and human-labeled ground truth.
- **Bias and Fairness:** Ensuring equal performance across dialects, languages, and demographics.

5.3 Test Cases

Example test cases include:

- **TC-01 Image Upload & OCR:** Upload a handwritten complaint image; check OCR accuracy.
- **TC-02 Multilingual Submission:** Submit grievances in local languages; verify accurate translation and tagging.
- **TC-03 Toxic Content Detection:** Detect abusive language; flag for manual review.
- **TC-04 API Failure Handling:** Simulate AI timeout; verify retry and fallback logic.
- **TC-05 Load Testing:** Simulate 1000 concurrent submissions; ensure system responsiveness.
- **TC-06 RAG Accuracy:** Verify correct retrieval of policy documents for specific schemes or rules.

Chapter 6

Implementation and Code

This chapter focuses on the logical implementation of the system's core features, especially the AI pipeline based on Retrieval Augmented Generation (RAG).

6.1 Implementation Logic: The RAG Pipeline

The RAG pipeline follows these stages:

1. **Ingestion:** Government policy documents, rulebooks, and FAQs (PDFs, text) are loaded. Text is extracted and split into manageable chunks.
2. **Embedding:** Each chunk is converted into a vector using an embedding model.
3. **Storage:** Vectors and metadata are stored in a vector database such as ChromaDB.
4. **Retrieval:** Grievance text is embedded and used to query the vector store for semantically similar chunks.
5. **Generation:** The grievance text and retrieved policy chunks are combined into a prompt and passed to the LLM, which generates a grounded response.

6.2 Python (FastAPI) — AI Classification Endpoint

Listing 6.1: FastAPI Endpoint for Grievance Analysis

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import uvicorn
from langchain_community.llms import Ollama

# Initialize FastAPI app
app = FastAPI()

# Initialize local LLM (Llama 3.2 via Ollama)
```

```
llm = Ollama(model="llama3.2")

# Define the input data model
class Grievance(BaseModel):
    text: str

@app.post("/analyze_grievance")
async def analyze(grievance: Grievance):
    try:
        # Construct the Prompt for Classification and Summarization
        prompt = f"""
        Act as a government grievance officer. Analyze the following
        grievance:

        "{grievance.text}"

        Perform the following tasks:
        1. Categorize it into one of these departments.
        2. Assess urgency: [High, Medium, Low] based on safety risks
        .
        3. Provide a 1-sentence summary for the dashboard.
        4. Draft a polite, empathetic reply to the citizen.

        Return the result in valid JSON format.
        """

        # Invoke the LLM
        response = llm.invoke(prompt)

        # In a real scenario, parse JSON string to a dictionary
        return {"analysis": response}

    except Exception as e:
        # Log the error and return a 500 status code
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    # Run the server on port 8000
    uvicorn.run(app, host="0.0.0.0", port=8000)
```


6.3 Node.js (Express) — Handling Submission

Listing 6.2: Node.js Controller for Grievance Submission

```
const express = require('express');
const axios = require('axios'); // Library for making HTTP requests
const app = express();

app.use(express.json()); // Middleware to parse JSON bodies

// Endpoint to handle grievance submission
app.post('/api/submit-grievance', async (req, res) => {
  const { userText, userId } = req.body;

  try {
    // Communication with Python Microservice
    const aiResponse = await axios.post(
      'http://localhost:8000/analyze_grievance',
      { text: userText }
    );

    // Extract analysis from the AI response
    const { category, urgency, summary } = aiResponse.data.analysis;

    // Logic to save to MongoDB (Pseudocode)
    // const newGrievance = new GrievanceModel({
    //   userId,
    //   text: userText,
    //   category,
    //   urgency,
    //   summary
    // });
    // await newGrievance.save();

    // Send success response to the frontend
    res.status(200).json({
      message: "Grievance Received Successfully",
      category_detected: category,
      urgency_level: urgency
    });
  }
});
```

```
    } catch (error) {
      console.error("AI Service Error", error);
      // Fallback Logic: If AI fails, save with default 'Pending'
      status
      // await saveGrievanceWithoutAI(userText, userId);
      res.status(500).send("Internal Server Error - Saved for manual
        review");
    }
  });

app.listen(3000, () => console.log('Node.js Server running on port
  3000'));
```

6.4 Deployment Strategy

The application deployment relies on Docker for containerization.

- **Development Environment:** Developers use Docker Desktop and `docker-compose` to spin up the stack (Node.js, Python, MongoDB) with a single command.
- **Production Environment:** Containers are deployed to GPU-optimized instances (for the AI service) and CPU-optimized instances (for Node.js and MongoDB) on platforms like AWS ECS or Google Cloud Run.

Chapter 7

Snapshots

This chapter provides a visualization of the User Interface (UI) and key components of the deployed AI Grievance Redressal System.

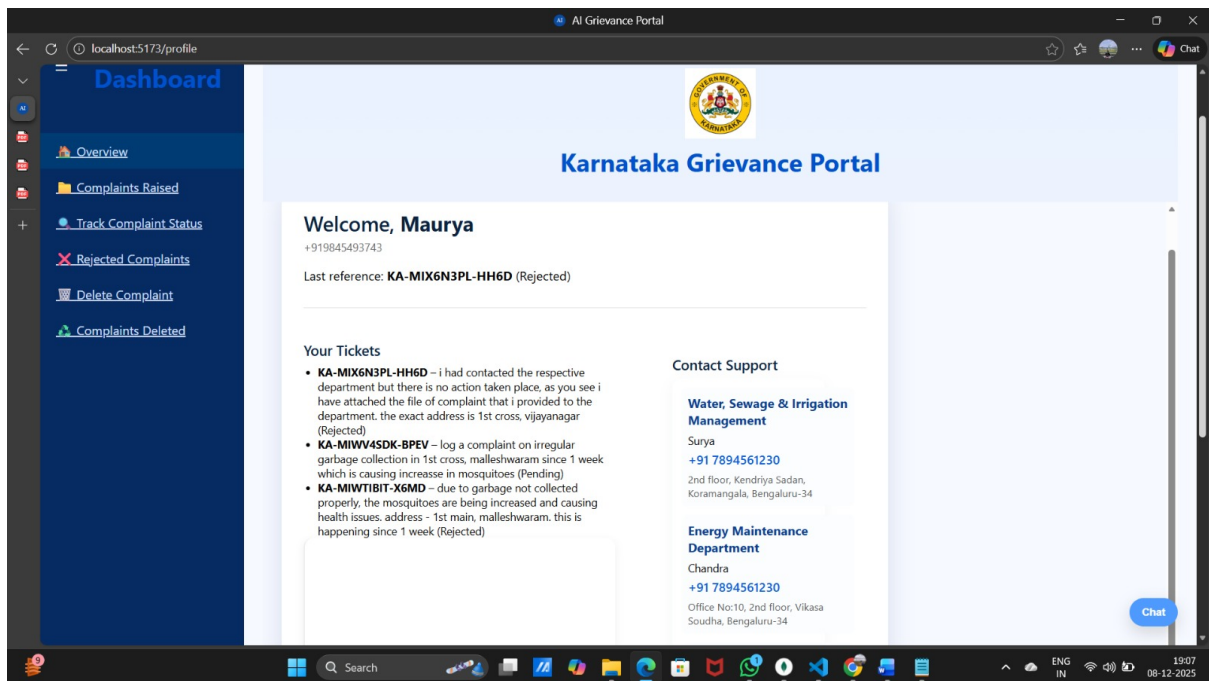


Figure 7.1: Citizen Dashboard showing welcome message and current tickets

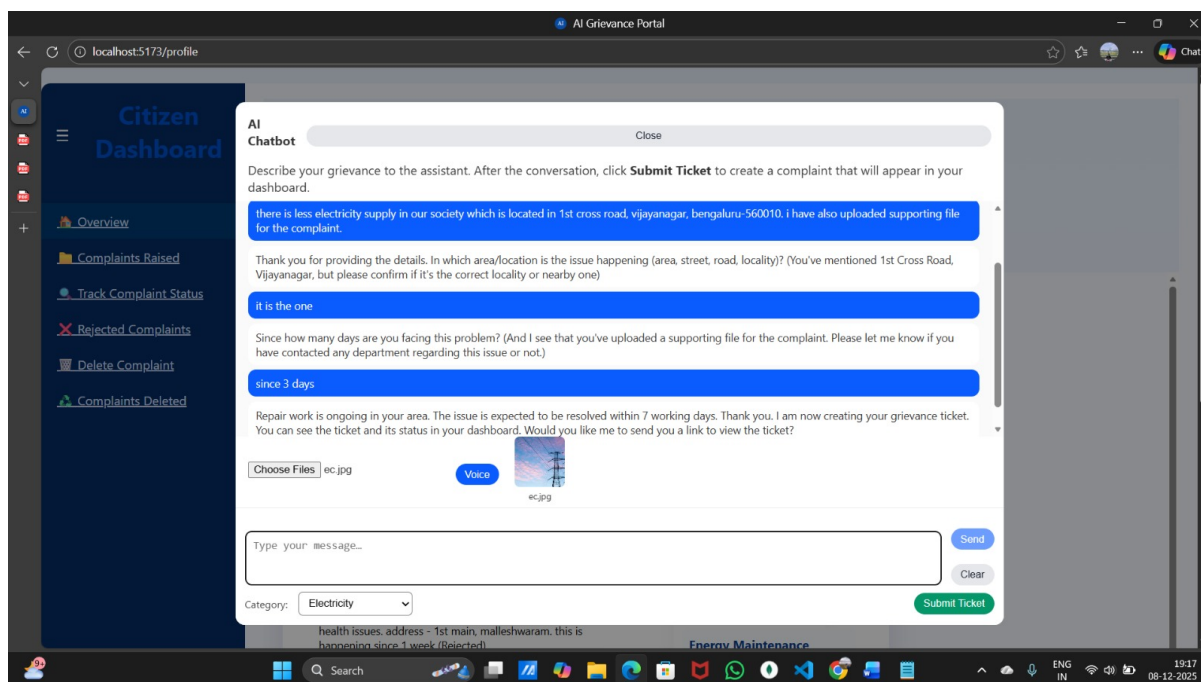


Figure 7.2: AI Chatbot interface collecting grievance details from the citizen

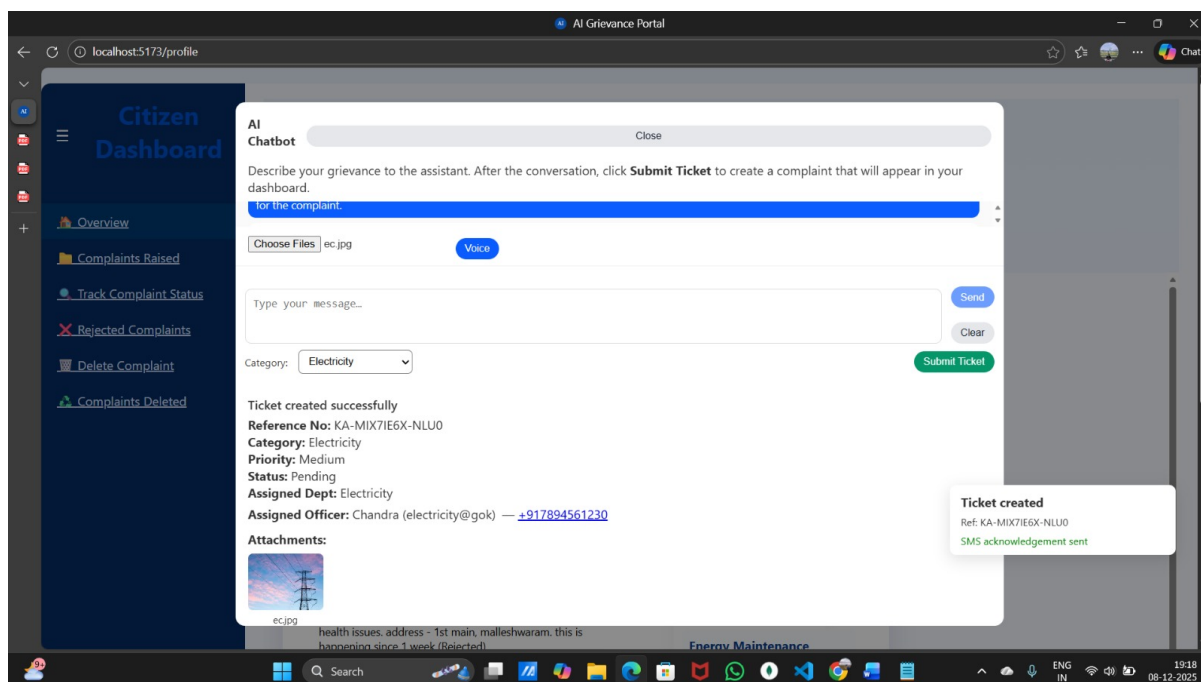


Figure 7.3: Chatbot confirming grievance creation with reference details

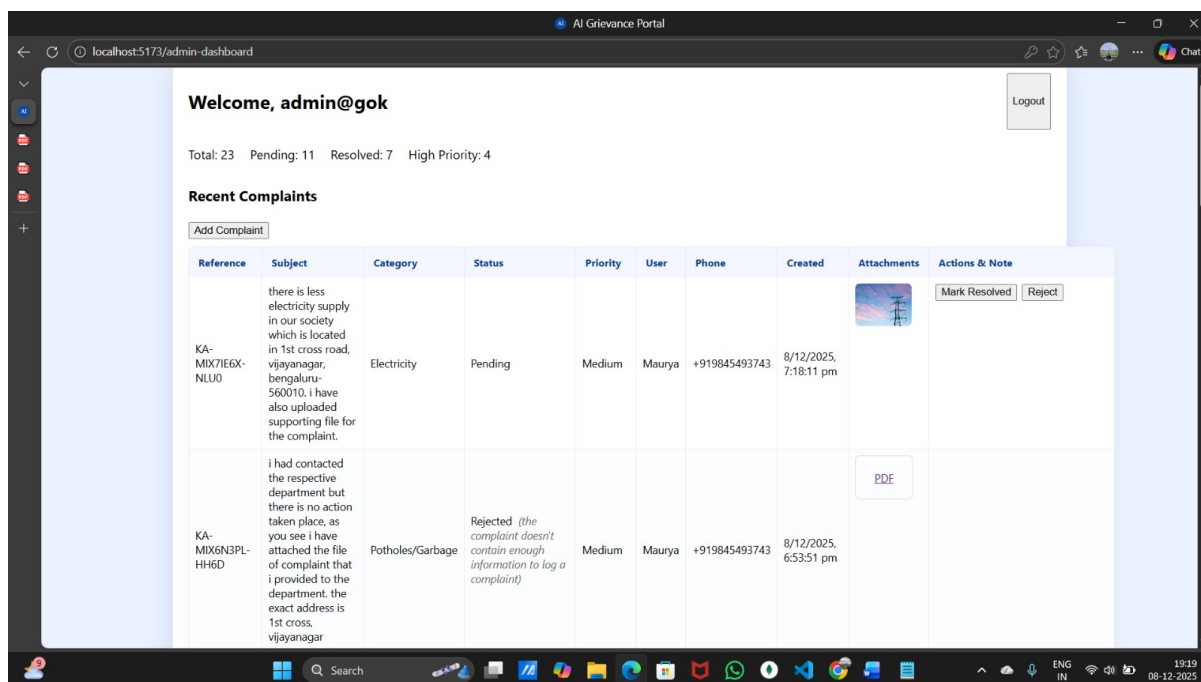


Figure 7.4: Admin Dashboard with recent complaints and status summary

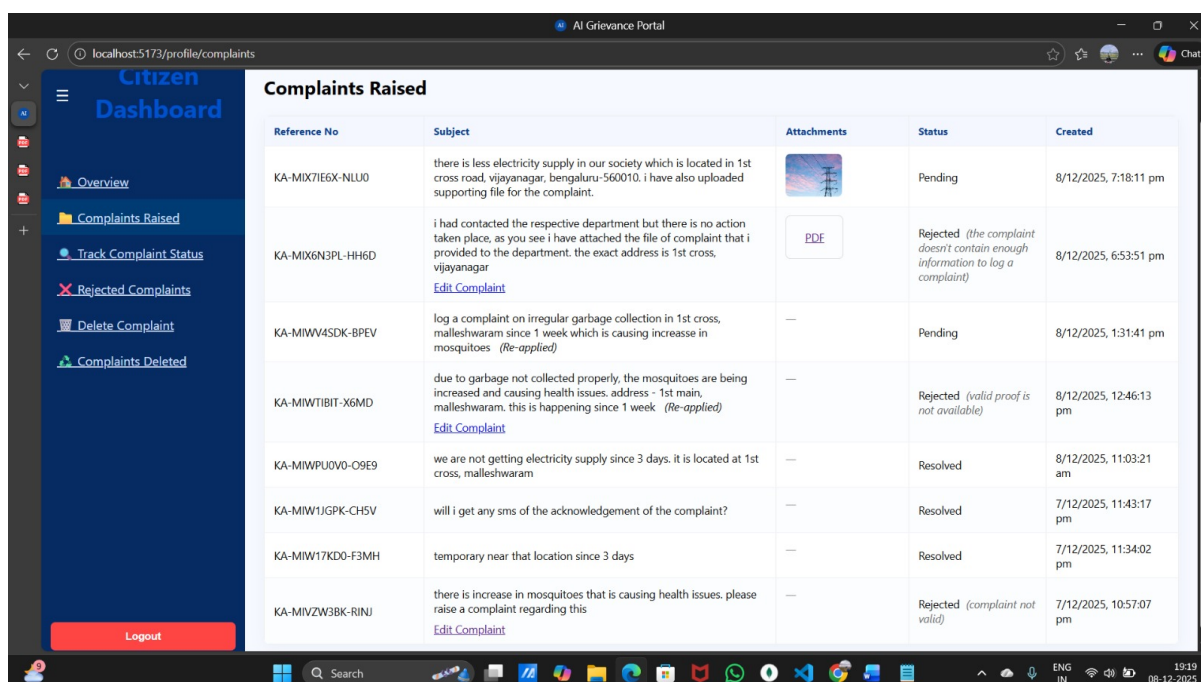


Figure 7.5: Citizen view of all complaints raised with their current status

Conclusion

The “AI Grievance Redressal System” project represents a significant technological leap in the administration of public and corporate services. By integrating cutting-edge Artificial Intelligence technologies—specifically Large Language Models (LLMs) and Computer Vision—the system successfully addresses the critical pain points of traditional grievance mechanisms: latency, inefficiency, and lack of transparency.

Summary of Achievements

- **Hybrid Architecture:** A robust hybrid architecture using Node.js for scalable I/O handling and Python for high-performance AI computation ensures that the system can handle high user traffic while performing complex inference tasks.
- **Deep Learning Integration:** The system demonstrates the capability of modern LLMs to understand, categorize, and summarize complex, unstructured grievances across multiple languages, significantly reducing the cognitive load on human officers.
- **Proactive Governance:** Beyond simple ticket logging, the system provides actionable insights through predictive analytics and heatmaps, empowering authorities to address issues proactively.

Limitations

- **Resource Intensity:** Local deployment of high-parameter LLMs requires significant GPU resources, which may be a cost constraint for smaller bodies.
- **Hallucination Risk:** Despite RAG and safeguards, LLMs maintain a non-zero risk of generating incorrect policy advice. Critical decisions must therefore remain under human supervision.

Future Scope

- **Voice Integration:** Implementing full-duplex speech interfaces for phone-call-based grievance lodging.

- **Blockchain Integration:** Creating an immutable, tamper-proof ledger of grievances and resolutions to enhance accountability.
- **Hyper-Local Training:** Fine-tuning models on local dialects and administrative jargon to improve accuracy in rural and regional deployments.

This project demonstrates that AI is not just a futuristic concept but a practical, deployable tool capable of humanizing governance, enhancing efficiency, and ultimately improving the quality of life for citizens.

Bibliography

- [1] W. Stallings, *Data and Computer Communications*, 10th ed. Pearson Education, 2014.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2020.
- [3] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. (Draft), Pearson, 2023.
- [4] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [5] FastAPI Documentation — “FastAPI: Modern, fast (high-performance) web framework for building APIs with Python.” <https://fastapi.tiangolo.com/>
- [6] Node.js Documentation — “Node.js: JavaScript runtime built on Chrome’s V8 engine.” <https://nodejs.org/>
- [7] React Documentation — “React: A JavaScript library for building user interfaces.” <https://reactjs.org/>
- [8] MongoDB Documentation — “MongoDB: The application data platform.” <https://www.mongodb.com/>
- [9] SQLite Documentation — “SQLite: Self-contained SQL database engine.” <https://sqlite.org/>
- [10] Ollama LLaMA Model — “Ollama: Run LLaMA and other LLMs locally.” <https://ollama.com/>
- [11] Twilio API Documentation — “Twilio SMS and messaging APIs.” <https://www.twilio.com/docs>
- [12] G. Booch, I. Jacobson, and J. Rumbaugh, *The Unified Modeling Language User Guide*, 2nd ed., Addison-Wesley, 2005.
- [13] R. Heeks, “Understanding e-Government for Development,” Institute for Development Policy and Management, 2001.
- [14] Y. Zhang, Q. Yang, “A Survey on Multi-Task Learning,” *IEEE Transactions on Knowledge and Data Engineering*, 2021.